1. Exact Go topics to prepare

Go concepts, technical answers and ten machine-coding implementations. Start at item A and use the numbered sections as independent chat topics. Python DSA is outside this reference.

A. Language fundamentals. var, :=, const, zero values, scope/shadowing; if, switch, for/range; functions, multiple returns, variadic parameters, closures. Code: word counter; config validation; a function that returns a value and an error.

B. Collections and data. Arrays vs slices; slice header, len/cap, append, copy, backing-array sharing; nil vs empty slices; maps, comma-ok, delete, nil maps, iteration order; strings, bytes/runes, Unicode. Code: independent slice copy and frequency map.

C. Types and API design. Pointers/value passing; structs, embedding, composition; methods and receiver selection; method sets; implicit interfaces; type assertions/type switches; typed nil; generics, any/comparable. Code: small store interface and two implementations.

D. Errors and resources. error interface; sentinel/custom errors; errors.Is/As; wrapping with %w; defer evaluation/order; panic/recover; resource ownership. Code: wrap a not-found error and close a file on every return path.

E. Concurrency. Goroutines; channels and direction; buffered/unbuffered, nil/closed behavior; select; producer/consumer, fan-out/fan-in; Mutex/RWMutex, WaitGroup, Once, atomics; deadlock/race/leak; bounded work and backpressure. Code: examples 1, 3, 4, 6, 7 and 10.

F. Context and time. Background/TODO; WithCancel/Timeout/Deadline; Done/Err; cancellation propagation; context values; time.Duration, timers/tickers; shutdown; retry/backoff/jitter/idempotency. Code: examples 1, 2, 8 and shutdown on page 11.

G. Backend Go. net/http, handlers, ServeMux, middleware, JSON/tags/validation, request/body limits, HTTP status codes; reusable clients, deadlines, body closure; io.Reader/Writer, streaming; database/sql, pooling, rows, transactions. Code: examples 2, 5, 9 and page 11.

H. Testing and tooling. testing.T, table-driven tests, subtests, t.Cleanup, handwritten fakes, httptest; deterministic time/cancellation tests; benchmarks/benchmem, fuzzing; gofmt, go test/vet, race detector, pprof. Code: page 11; add the failure cases printed beside each example.

I. Practical depth. Modules/packages, exports, go.mod/go.sum, go mod tidy, build tags; dependency injection; allocation/escape analysis, stack/heap, GC, goroutine scheduler and GOMAXPROCS; profiling; cgo pointer/callback lifetime for your SQLite work. Explain: one actual performance or lifecycle failure.

J. Your domain knowledge. Kubernetes objects, CRDs, manager/client/cache, reconcile/watch/index/predicate, owner references, finalizers/status/requeue/leader election; SQLite callbacks and driver connections; Raft context. Read: the exact resume-focused paths on page 10.

Exact reading: Go Tour: Basics, Methods and interfaces, Concurrency (https://go.dev/tour/list). Then look up the APIs used by the example in https://pkg.go.dev/std. For language details: https://go.dev/ref/spec.

2. Go technical round: values and types

The answers are deliberately short. The code gives you something concrete to explain and modify.

Does Go pass by reference? No. Arguments are values; a copied pointer still points to the same object.

What is copied for a slice? Its descriptor. Elements can still share the backing array.

Can append affect another slice? Yes, if shared capacity is reused; reallocation changes that sharing.

nil map versus nil slice? A nil map cannot be assigned into. A nil slice can be appended to.

Value or pointer receiver? Use a pointer for mutation or identity; also consider copying cost and method sets.

How are interfaces satisfied? Implicitly by a matching method set.

Why a non-nil interface with nil data? The interface retains a concrete dynamic type.

What does defer capture? Call arguments are evaluated now; deferred calls execute later, in reverse order.

What does errors.Is do? Checks an error/wrapping chain for a target; errors.As finds an assignable error type.

When use panic? For exceptional invariant failures, not ordinary validation or network errors.

What is a rune? An alias of int32 used for Unicode code points; one visible character may contain several.

What is time.Duration? An int64 nanosecond count. Write 2*time.Second rather than an unexplained integer.

Predict these results

```
// Each block goes inside main.
a := []int{1, 2, 3}
b := a[:2]
b[0] = 9
fmt.Println(a) // [9 2 3]
b = append(b, 8)
fmt.Println(a) // [9 2 8]
copyB := append([]int(nil), a...)
copyB[0] = 7 // a is unchanged.
```

Practice: give a capacity-limited slice to append. Explain why replacing a slice variable inside a function does not replace the caller’s variable.

```
type Problem struct{}
func (*Problem) Error() string { return "bad" }
func problem() error {
 var p *Problem
 return p
}
// In main: fmt.Println(problem() == nil)
// Prints false: type=*Problem, value=nil.
```

```
x := 1
defer fmt.Println(x)             // Prints 1.
defer func() { fmt.Println(x) }() // Prints 2.
x = 2
// Closure runs first (LIFO).
```

```
type Counter struct { N int }
func (c *Counter) Inc() { c.N++ }
// main: c := Counter{}; c.Inc()
// *Counter satisfies interface{ Inc() }.
// Counter does not; the method set differs.
```

Version trap: Go 1.22 language behavior gives loop-declared variables per-iteration identity. Preexisting variables assigned in the loop still differ. Check go.mod before interpreting an old closure question.

3. Go technical round: concurrency and I/O

Goroutine versus OS thread? A goroutine is runtime-managed work; many goroutines can run over OS threads.

Concurrency versus parallelism? Concurrency structures overlapping tasks. Parallelism runs tasks at the same time.

When does channel send block? Unbuffered: until paired with receive. Buffered: when no capacity is available.

Receive from a closed channel? Buffered values first, then zero value and ok=false. Sending or closing again panics.

What about a nil channel? Send/receive block indefinitely; its select cases cannot proceed.

Who should close a channel? The component that can prove no more sends are possible; with multiple senders, coordinate them first.

Does cancellation kill a goroutine? No. Code and blocking operations must cooperate with cancellation.

Race versus deadlock versus leak? Unsynchronized conflicting access; progress blocked; unnecessary goroutine never exits.

Mutex versus RWMutex? Choose based on the invariant and measured workload. RWMutex is not automatically faster.

WaitGroup order? Register work before launching it; each registered task must finish. Do not copy synchronization values.

Does race-free mean correct? No. Deadlocks, leaks and incorrect check-then-act behavior may remain.

Why bounded concurrency? It limits load on memory, sockets, APIs and databases. Unbounded goroutines can overwhelm them.

Why propagate request context? So downstream I/O can respect the caller’s cancellation and deadline.

Why call the cancel function? Release resources associated with derived contexts when the work ends.

What belongs in context values? Request-scoped metadata such as a trace ID; not dependency configuration.

Why reuse an HTTP client? Reuse its transport/connections. Set suitable timeouts and always release response bodies.

Can closing a body alone ensure reuse? Not always; unread HTTP/1 response data can prevent connection reuse. Bound any reading/draining.

Is sql.DB one connection? It manages a connection pool. Pool sizing, deadlines and leaked rows affect capacity.

How test concurrent behavior? Use signals/barriers and assert invariants; sleeps are not synchronization.

What does -race miss? Unexecuted paths and non-race correctness bugs. Test relevant interleavings.

What is escape analysis? Compiler reasoning about value lifetime determines allocation; a pointer does not automatically imply heap allocation.

What do GC and GOMAXPROCS control? GC manages unreachable memory; GOMAXPROCS limits simultaneous execution of Go code, not the total goroutine count.

CPU/heap/goroutine profile? Use CPU for hot paths, heap for allocations/retention and goroutine stacks for blocked or leaking work.

Why retries need idempotency? A timeout can occur after a remote side effect succeeded. Retrying a write may duplicate it.

Read: https://pkg.go.dev/context • https://pkg.go.dev/sync • https://go.dev/ref/mem • https://go.dev/doc/articles/race_detector • https://go.dev/doc/diagnostics

4. Machine coding: pool and URL checker

All ten examples use package study and standard-library dependencies. Save each block under its named .go file in one folder, create go.mod with module example.com/go-study and go 1.22, then use the tests on page 11. Examples 2 and 6 reuse Pool. These are teaching implementations with stated limits.

1. Bounded worker pool

Process integers with at most n workers; return ordered values/errors. Cancellation discards partial results.

```
package study
import (
  "context"
  "errors"
  "sync"
)
type Result struct {
  Value int
  Err   error
}
// fn must honor ctx for prompt cancellation.
func Pool(ctx context.Context, n int,
  input []int,
  fn func(context.Context, int) (int, error),
) ([]Result, error) {
  if n < 1 {
    return nil, errors.New("workers")
  }
  out := make([]Result, len(input))
  jobs := make(chan int)
  var wg sync.WaitGroup
  for w := 0; w < n; w++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      for {
        select {
        case <-ctx.Done():
          return
        case i, ok := <-jobs:
          if !ok {
            return
          }
          if ctx.Err() != nil {
            return
          }
          v, err := fn(ctx, input[i])
          out[i] = Result{v, err}
        }
      }
    }()
  }
send:
  for i := range input {
    select {
    case <-ctx.Done():
      break send
    case jobs <- i:
    }
  }
  close(jobs) // Sole sender owns closure.
  wg.Wait()   // Finish writes before returning.
  if err := ctx.Err(); err != nil {
    return nil, err // Discard partial results.
  }
  return out, nil
}
```

Explain: workers write distinct result indexes; Wait prevents returning while writes continue. fn must be concurrency-safe and honor ctx.
Change: make it fail-fast on the first job error.
Test: n=0, empty input, job failure, cancellation during work.

2. Concurrent URL checker

Check URLs with a shared client and worker limit; preserve input order. HTTP 500 is a status, not a transport error.

```
package study
import (
  "context"
  "net/http"
  "time"
)
type Check struct {
  URL     string
  Status  int
  Elapsed time.Duration
  Err     error
}
// Uses Pool from pool.go. Results stay ordered.
func CheckURLs(ctx context.Context,
  urls []string, n int) ([]Check, error) {
  client := &http.Client{
    Timeout: 2 * time.Second,
  }
  indexes := make([]int, len(urls))
  out := make([]Check, len(urls))
  for i := range indexes {
    indexes[i] = i
  }
  results, err := Pool(ctx, n, indexes,
    func(ctx context.Context, i int) (int, error) {
      start := time.Now()
      out[i].URL = urls[i]
      req, e := http.NewRequestWithContext(
        ctx, http.MethodGet, urls[i], nil)
      if e == nil {
        var resp *http.Response
        resp, e = client.Do(req)
        if e == nil {
          out[i].Status = resp.StatusCode
          resp.Body.Close()
        }
      }
      out[i].Elapsed = time.Since(start)
      return i, e
    })
  if err != nil {
    return nil, err
  }
  for i, r := range results {
    out[i].Err = r.Err
  }
  return out, nil
}
```

Explain: context, client timeout and body closure. This version closes without draining; connection reuse may suffer.
Change: inject a client and add bounded draining.
Test: local success, 503, malformed URL, slow response.

5. Machine coding: cache and rate limiter

3. Concurrent TTL cache

String keys/values. TTL zero means no expiry; negative is invalid; expiration is checked on Get.

```
package study
import (
  "errors"
  "sync"
  "time"
)
type entry struct {
  value  string
  expiry time.Time
}
type Cache struct {
  mu   sync.Mutex
  data map[string]entry
  now  func() time.Time
}
func NewCache(now func() time.Time) *Cache {
  return &Cache{data: map[string]entry{}, now: now}
}
// Zero TTL means no expiry; negative is invalid.
func (c *Cache) Set(k, v string,
  ttl time.Duration) error {
  if ttl < 0 {
    return errors.New("negative TTL")
  }
  c.mu.Lock()
  defer c.mu.Unlock()
  e := entry{value: v}
  if ttl > 0 {
    e.expiry = c.now().Add(ttl)
  }
  c.data[k] = e
  return nil
}
func (c *Cache) Get(k string) (string, bool) {
  c.mu.Lock()
  defer c.mu.Unlock()
  e, ok := c.data[k]
  if !ok {
    return "", false
  }
  if !e.expiry.IsZero() &&
    !c.now().Before(e.expiry) {
    delete(c.data, k)
    return "", false
  }
  return e.value, true
}
func (c *Cache) Delete(k string) {
  c.mu.Lock()
  defer c.mu.Unlock()
  delete(c.data, k)
}
```

Explain: protect the full lookup/expiry invariant. Inject time rather than sleeping.
Change: add a capacity limit or stoppable cleanup.
Test: exact expiry boundary, overwrite, delete, concurrent calls.
Limit: expired unused keys remain until access; no size bound.

4. Per-key token bucket

Allow(key) returns immediately. rate is tokens/second; burst is capacity. Each new key starts full.

```
package study
import (
  "errors"
  "sync"
  "time"
)
type bucket struct {
  tokens float64
  last   time.Time
}
type Limiter struct {
  mu          sync.Mutex
  rate, burst float64
  states      map[string]bucket
  now         func() time.Time
}
func NewLimiter(rate, burst int,
  now func() time.Time) (*Limiter, error) {
  if rate < 1 || burst < 1 {
    return nil, errors.New("invalid limits")
  }
  return &Limiter{rate: float64(rate),
    burst: float64(burst), now: now,
    states: map[string]bucket{}}, nil
}
func (l *Limiter) Allow(key string) bool {
  l.mu.Lock()
  defer l.mu.Unlock()
  now := l.now()
  b, ok := l.states[key]
  if !ok {
    b = bucket{l.burst, now}
  }
  elapsed := now.Sub(b.last).Seconds()
  if elapsed < 0 {
    elapsed = 0
  }
  b.tokens += elapsed * l.rate
  if b.tokens > l.burst {
    b.tokens = l.burst
  }
  b.last = now
  allowed := b.tokens >= 1
  if allowed {
    b.tokens--
  }
  l.states[key] = b
  return allowed
}
```

Explain: elapsed-time refill plus token consumption occurs under one lock. Use a nondecreasing clock.
Change: evict inactive keys and expose rejection metrics.
Test: burst, refill, key isolation, invalid limits.
Limit: unbounded key map; process-local enforcement.

6. Machine coding: Task REST API

5. Create/list/get/delete tasks. Uses Go 1.22+ method-aware ServeMux. In-memory IDs/state; list order unspecified. Reject unknown fields, empty title, oversized input and extra JSON values. Left column continues into right column.

```
package study
import (
  "encoding/json"
  "io"
  "net/http"
  "strconv"
  "strings"
  "sync"
)
type Task struct {
  ID    int    `json:"id"`
  Title string `json:"title"`
}
func reply(w http.ResponseWriter,
  status int, value any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(value)
}
func TaskAPI() http.Handler {
  var mu sync.Mutex
  tasks := map[int]Task{}
  next := 0
  mux := http.NewServeMux()
  mux.HandleFunc("POST /tasks",
    func(w http.ResponseWriter, r *http.Request) {
      defer r.Body.Close()
      r.Body = http.MaxBytesReader(w, r.Body, 4096)
      var input struct {
        Title string `json:"title"`
      }
      dec := json.NewDecoder(r.Body)
      dec.DisallowUnknownFields()
      err := dec.Decode(&input)
      if err == nil {
        err = dec.Decode(new(any))
        if err == io.EOF {
          err = nil
        } else if err == nil {
          err = io.ErrUnexpectedEOF
        }
      }
      title := strings.TrimSpace(input.Title)
      if err != nil || title == "" {
        reply(w, 400, "invalid task")
        return
      }
      mu.Lock()
      next++
      task := Task{next, title}
      tasks[next] = task
      mu.Unlock()
      reply(w, 201, task)
    })
  mux.HandleFunc("GET /tasks",
    func(w http.ResponseWriter, r *http.Request) {
      mu.Lock()
      list := make([]Task, 0, len(tasks))
      for _, t := range tasks {
        list = append(list, t)
      }
      mu.Unlock()
      reply(w, 200, list)
    })
  getOrDelete := func(w http.ResponseWriter,
    r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id"))
    if err != nil || id < 1 {
      reply(w, 400, "invalid ID")
      return
    }
    mu.Lock()
    task, ok := tasks[id]
    if ok && r.Method == http.MethodDelete {
      delete(tasks, id)
    }
    mu.Unlock()
    if !ok {
      reply(w, 404, "not found")
      return
    }
    if r.Method == http.MethodDelete {
      w.WriteHeader(204)
      return
    }
    reply(w, 200, task)
  }
  mux.HandleFunc("GET /tasks/{id}", getOrDelete)
  mux.HandleFunc("DELETE /tasks/{id}", getOrDelete)
  return mux
}
```

Run: attach TaskAPI() to http.Server; shutdown example is on page 11.
Example requests: POST /tasks with {"title":"fix bug"}; GET /tasks/1; DELETE /tasks/1.
Explain: locks protect state, not response writes. The decoder accepts exactly one object. Response encoding errors are ignored here after headers are sent.
Change: add a store interface, sorted pagination, or a SQL implementation.
Test: malformed/extra JSON, missing task, wrong method, body limit and concurrent creates.
Limits: no persistence/authentication; JSON errors are simple strings; all invalid bodies return 400.

7. Machine coding: jobs and pub/sub

6. Bounded batch job executor

Validate job IDs and run a finite batch using Pool; return queued/running/terminal state outcomes.

```
package study
import (
  "context"
  "errors"
  "sync"
)
type Job struct {
  ID  string
  Run func(context.Context) error
}
// Finite batch executor; uses Pool from pool.go.
func Execute(ctx context.Context, n int,
  jobs []Job) (map[string]string, error) {
  if n < 1 {
    return nil, errors.New("workers")
  }
  states := map[string]string{}
  indexes := make([]int, len(jobs))
  for i, j := range jobs {
    if j.ID == "" || j.Run == nil {
      return nil, errors.New("invalid job")
    }
    if _, ok := states[j.ID]; ok {
      return nil, errors.New("duplicate ID")
    }
    states[j.ID] = "queued"
    indexes[i] = i
  }
  var mu sync.Mutex
  _, err := Pool(ctx, n, indexes,
    func(ctx context.Context, i int) (int, error) {
      j := jobs[i]
      mu.Lock()
      states[j.ID] = "running"
      mu.Unlock()
      e := j.Run(ctx) // No lock across work.
      state := "succeeded"
      if e != nil {
        state = "failed"
      }
      if errors.Is(e, context.Canceled) ||
        errors.Is(e, context.DeadlineExceeded) {
        state = "canceled"
      }
      mu.Lock()
      states[j.ID] = state
      mu.Unlock()
      return i, e
    })
  if err != nil {
    // All workers have finished before this loop.
    for id, state := range states {
      if state == "queued" {
        states[id] = "canceled"
      }
    }
  }
  return states, err
}
```

Explain: work runs outside the state lock. Completion is returned after all workers finish. Job functions must honor cancellation.
Change: add Submit/Status methods and bounded pending capacity to make it a background scheduler.
Test: duplicates, failed jobs, queued cancellation.
Limit: batch execution; no live status API, scheduling times or persistence.

7. In-memory pub/sub

Subscribe, publish, unsubscribe, close. Each subscriber has buffer 1; a full buffer drops the newest message.

```
package study
import "sync"
type Bus struct {
  mu     sync.Mutex
  topics map[string]map[int]chan string
  next   int
  closed bool
}
func NewBus() *Bus {
  return &Bus{
    topics: map[string]map[int]chan string{},
  }
}
func (b *Bus) Subscribe(
  topic string,
) (<-chan string, func()) {
  b.mu.Lock()
  defer b.mu.Unlock()
  ch := make(chan string, 1)
  if b.closed {
    close(ch)
    return ch, func() {}
  }
  b.next++
  id := b.next
  if b.topics[topic] == nil {
    b.topics[topic] = map[int]chan string{}
  }
  b.topics[topic][id] = ch
  return ch, func() {
    b.mu.Lock()
    defer b.mu.Unlock()
    if c, ok := b.topics[topic][id]; ok {
      delete(b.topics[topic], id)
      close(c)
      if len(b.topics[topic]) == 0 {
        delete(b.topics, topic)
      }
    }
  }
}
// Drop newest message if a subscriber is full.
func (b *Bus) Publish(topic, msg string) int {
  b.mu.Lock()
  defer b.mu.Unlock()
  dropped := 0
  for _, ch := range b.topics[topic] {
    select {
    case ch <- msg:
    default:
      dropped++
    }
  }
  return dropped
}
func (b *Bus) Close() {
  b.mu.Lock()
  defer b.mu.Unlock()
  if b.closed {
    return
  }
  b.closed = true
  for _, subs := range b.topics {
    for _, ch := range subs {
      close(ch)
    }
  }
  b.topics = map[string]map[int]chan string{}
}
```

Explain: publish/unsubscribe/close share a lock, so no send races with channel closure. Unsubscribe is idempotent.
Change: configurable buffer and explicit slow-consumer policy.
Test: full subscriber, unsubscribe twice, concurrent publish/close.
Limit: lossy delivery, process-local, topic fan-out holds one lock.

8. Machine coding: retries and log processing

8. Cancellable retry

Retry selected errors, capped exponential backoff; total attempts includes the first call. Parent ctx controls the overall deadline.

```
package study
import (
  "context"
  "errors"
  "time"
)
// attempts includes the initial call.
func Retry(ctx context.Context, attempts int,
  op func(context.Context) error,
  retryable func(error) bool) error {
  if attempts < 1 {
    return errors.New("attempts")
  }
  delay := 10 * time.Millisecond
  for i := 0; i < attempts; i++ {
    if err := ctx.Err(); err != nil {
      return err
    }
    err := op(ctx)
    if err == nil {
      return nil
    }
    if i == attempts-1 || !retryable(err) {
      return err
    }
    timer := time.NewTimer(delay)
    select {
    case <-ctx.Done():
      timer.Stop()
      return ctx.Err()
    case <-timer.C:
    }
    delay *= 2
    if delay > time.Second {
      delay = time.Second
    }
  }
  return nil
}
```

Example: Retry(ctx, 3, operation, isTemporary).
Explain: context does not interrupt a noncooperating operation. A retry policy must account for side effects.
Change: inject waiting/time; add jitter and per-attempt timeout.
Test: success first try, permanent error, exhaustion, cancel during backoff.
Limit: no jitter; caller supplies error classification.

9. Streaming JSON-line logs

Read records such as {"level":"info"}; count by level and write JSON. A malformed line fails the operation.

```
package study
import (
  "bufio"
  "encoding/json"
  "fmt"
  "io"
)
// Malformed lines fail; max line is 1 MiB.
func Summarize(r io.Reader, w io.Writer) error {
  scanner := bufio.NewScanner(r)
  scanner.Buffer(make([]byte, 4096), 1<<20)
  counts := map[string]int{}
  line := 0
  for scanner.Scan() {
    line++
    var record struct {
      Level string `json:"level"`
    }
    if err := json.Unmarshal(scanner.Bytes(),
      &record); err != nil {
      return fmt.Errorf("line %d: %w", line, err)
    }
    if record.Level == "" {
      return fmt.Errorf(
        "line %d: missing level", line)
    }
    counts[record.Level]++
  }
  if err := scanner.Err(); err != nil {
    return err
  }
  return json.NewEncoder(w).Encode(counts)
}
```

Explain: Reader/Writer dependencies allow files, strings or network streams. Check Scanner.Err and propagate writer errors.
Change: skip bad lines with a count, or filter by duration.
Test: empty file, malformed JSON, missing level, too-long line, reader/writer failure.
Limit: line size limited to 1 MiB; counts can grow with distinct levels.

9. Machine coding: reconciliation queue

10. Coalescing work queue

Duplicate pending keys collapse; an Add during processing causes another pass. Run supports exactly one worker and stops on error.

```
package study
import (
  "context"
  "sync"
)
type Queue struct {
  mu      sync.Mutex
  pending map[string]bool
  wake    chan struct{}
}
func NewQueue() *Queue {
  return &Queue{pending: map[string]bool{},
    wake: make(chan struct{}, 1)}
}
func (q *Queue) Add(key string) {
  q.mu.Lock()
  q.pending[key] = true
  q.mu.Unlock()
  select {
  case q.wake <- struct{}{}:
  default:
  }
}
func (q *Queue) pop() (string, bool) {
  q.mu.Lock()
  defer q.mu.Unlock()
  for key := range q.pending {
    delete(q.pending, key)
    return key, true
  }
  return "", false
}
// One Run caller only; reconcile re-reads state.
// An error stops Run; caller chooses retry policy.
func (q *Queue) Run(ctx context.Context,
  reconcile func(context.Context, string) error,
) error {
  for {
    if err := ctx.Err(); err != nil {
      return err
    }
    key, ok := q.pop()
    if ok {
      if err := reconcile(ctx, key); err != nil {
        return err
      }
      continue
    }
    select {
    case <-ctx.Done():
      return ctx.Err()
    case <-q.wake:
    }
  }
}
```

Explain: queue notification is separate from queued state. The wake buffer can coalesce signals because Run drains state.
Change: add per-key retry/backoff without concurrent work on the same key.
Test: repeated key, Add during reconcile, error and cancellation.
Limit: no fairness, capacity bound or automated retries. Failed popped key must be re-added by caller.

Go exercise tied to Secret rotation

Illustrative data-only checksum helper; not copied from your contribution. JSON sorts string map keys and encodes byte values, giving a stable input for this representation.

```
package study
import (
 "crypto/sha256"
 "encoding/hex"
 "encoding/json"
)
func SecretHash(
 data map[string][]byte,
) (string, error) {
 if data == nil { data = map[string][]byte{} }
 encoded, err := json.Marshal(data)
 if err != nil { return "", err }
 sum := sha256.Sum256(encoded)
 return hex.EncodeToString(sum[:]), nil
}
```

Practice: show unchanged hashes when insertion order changes; changed hashes when values change; no rollout from metadata edits. Explain whether hashing all keys should roll workloads after an unrelated key changes.

Controller translation: an event enqueues a resource key; reconcile reads current objects, computes desired state, updates only when needed and returns. A repeated call should converge.

Secret updates: a changed Pod-template annotation triggers a Deployment rollout. Environment-variable Secret updates do not refresh existing processes automatically.

Event identity: name alone can collide across kinds/namespaces. Inspect the actual fix and discuss UID as a separate strengthening choice.

Copy into a new chat: "Explain example 10 line by line, explain every concept it uses, then ask me to change its error policy."

10. Understand your project and OSS work

Use the real repository code for the items below. The helper examples teach concepts; they do not establish your project's exact implementation or PR status.

rqlite: query logging. Read request -> Store -> DB -> driver initialization -> SQLite trace callback -> statement tracking -> logger. Study STMT versus PROFILE, statement identity and lifetime, per-connection configuration, mutex protection, duration threshold, expanded SQL and cleanup. PROFILE reports completion timing; do not assume its value is an exact benchmark. Answer: Why pair events? What happens on re-execution/error/close? Can logs reveal secrets? What defaults stay unchanged?

rqlite: driver composition and flagforge. Trace configuration propagation and connection setup: extensions, checkpoints and tracing must coexist. Study database/sql driver registration and cgo callback/pointer rules as used by the actual driver. For flagforge, trace hide metadata into Go/Markdown/HTML generators. Answer: What is connection-specific? What happens on reconnect? Does hiding affect runtime availability or only generated output?

rqlite architecture prerequisite. Raft terms: leader/follower/candidate, terms, election, log replication, majority quorum, commit, snapshots and read consistency. Answer: With three voting nodes, what failures can preserve a majority? Your query-logging contribution does not mean you implemented Raft.

K8sGPT: Secret rotation. Read field-index registration -> Secret watch/mapping -> affected CR reconciliation -> data checksum -> Deployment Pod-template annotation. Study namespace-scoped lookup, deterministic hashing, irrelevant updates, shared/missing Secrets and deletion cleanup. Answer: Why a watch? Why an index? Why data-only? Why does the Deployment roll?

K8sGPT: Service diagnostics and Helm. For diagnostics, reproduce same-named resources and inspect Event identity filtering. For kube-rbac-proxy, compare Helm rendering with the flag enabled/disabled: containers, Service, ServiceMonitor and RBAC. Answer: What remains the default? What security changes with direct HTTP metrics?

KAI Scheduler: resource allocation. Read the allocation function and its regression test. Study quota/deserved-share units, queue priority, fractional resources and rounding remainder; work one numerical example by hand. Answer: What exact policy is intended? Which test proves your changed branch? How did maintainer feedback affect that interpretation?

Incident Investigator: complete path. Read startup/manager -> registered watches -> reconcile -> workload ownership -> evidence -> diagnosis rule -> correlation -> IncidentReport update -> recovery. Name real files/functions. Study Pod/ReplicaSet/Deployment, Job/CronJob, StatefulSet/DaemonSet, Events, logs, Node/PVC evidence, limits/timeouts and partial errors.

Incident Investigator: explain the decisions. CRD/custom resource, client/cache, scheme, idempotency, status vs spec, generation/resourceVersion, owner references, finalizers if used, requeue, predicates/indexes and leader election. Answer: What prevents duplicates? What survives restart? Why CRD instead of a DB? How is confidence defined? Which evidence can mislead a rule?

Tests and release. Pure-rule unit tests, fake-client behavior, envtest API semantics, real-cluster lifecycle tests. envtest lacks kubelet and normal controllers, so it cannot prove real Pod rollout. Understand Helm OCI, CRD uninstall behavior, RBAC, amd64/arm64 builds, GHCR and tag-based CI. Answer: Which test proves which claim?

Read: https://sqlite.org/c3ref/c_trace.html • https://kubernetes.io/docs/concepts/architecture/controller/ • https://pkg.go.dev/sigs.k8s.io/controller-runtime • https://book.kubebuilder.io/reference/envtest

11. Tests, SQL and graceful shutdown

Put this in study_test.go

```
package study
import (
 "context"
 "net/http/httptest"
 "strings"
 "testing"
 "time"
)
func TestPoolOrder(t *testing.T) {
 got, err := Pool(context.Background(), 2,
  []int{3, 1},
  func(_ context.Context, x int) (int, error) {
   return x*x, nil
  })
 if err != nil || got[0].Value != 9 ||
  got[1].Value != 1 { t.Fatal(got, err) }
}
func TestCacheExpiry(t *testing.T) {
 now := time.Unix(100, 0)
 c := NewCache(func() time.Time { return now })
 if err := c.Set("x", "v", time.Second);
  err != nil { t.Fatal(err) }
 now = now.Add(time.Second)
 if _, ok := c.Get("x"); ok {
  t.Fatal("must expire at boundary")
 }
}
func TestCreateTask(t *testing.T) {
 for _, tc := range []struct {
  body string; status int
 }{
  {`{"title":"read Go"}`, 201},
  {`{"title":""}`, 400},
  {`{"title":"a"} {}`, 400},
 } {
  w := httptest.NewRecorder()
  r := httptest.NewRequest("POST", "/tasks",
   strings.NewReader(tc.body))
  TaskAPI().ServeHTTP(w, r)
  if w.Code != tc.status { t.Fatal(w.Code) }
 }
}
```

Run: go test ./...; go test -race ./...; go vet ./...
Practice: add a cancellation test using channels, a local HTTP failure test using httptest.NewServer, and a benchmark. Race tests exercise only paths actually run.

Read the named APIs:
https://pkg.go.dev/testing
https://pkg.go.dev/net/http/httptest
https://pkg.go.dev/database/sql
https://pkg.go.dev/net/http

SQL lifecycle pattern (PostgreSQL)

```
// Imports: context, database/sql.
func Save(ctx context.Context, db *sql.DB,
 title string) error {
 tx, err := db.BeginTx(ctx, nil)
 if err != nil { return err }
 defer tx.Rollback() // Harmless after Commit.
 _, err = tx.ExecContext(ctx,
  "INSERT INTO tasks(title) VALUES ($1)",
  title)
 if err != nil { return err }
 return tx.Commit()
}
```

Know: db is a pool; configure its bounds. For queries: close rows, check Scan errors and rows.Err. SQL placeholders/driver registration depend on the database. This pattern needs an actual PostgreSQL driver/schema to run.

Serve TaskAPI and stop cleanly

```
// main.go in a separate cmd/server folder.
package main
import (
 "context"
 "errors"
 "log"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"
 study "example.com/go-study"
)
func main() {
 ctx, stop := signal.NotifyContext(
  context.Background(), os.Interrupt,
  syscall.SIGTERM)
 defer stop()
 srv := &http.Server{Addr: ":8080",
  Handler: study.TaskAPI(),
  ReadHeaderTimeout: 5*time.Second}
 done := make(chan struct{})
 go func() {
  defer close(done)
  <-ctx.Done()
  shutdown, cancel := context.WithTimeout(
   context.Background(), 5*time.Second)
  defer cancel()
  if err := srv.Shutdown(shutdown); err != nil {
   log.Print(err); _ = srv.Close()
  }
 }()
 err := srv.ListenAndServe()
 if !errors.Is(err, http.ErrServerClosed) {
  log.Fatal(err)
 }
 <-done // Wait for graceful shutdown to finish.
}
```

Explain: ListenAndServe returns when shutdown starts; main waits on done until Shutdown/Close completes. The fresh shutdown context is not already canceled. This example does not stop arbitrary background work; those workers need their own lifecycle.